

Stopword Impact Analysis on NLP Tasks

Using the Reuters-21578 Newswire Corpus

Submitted by

BABHALE ROHAN LAXMIKANT 23BCS026

Under the guidance of:

Prof. Krishnendu Ghosh



Natural Language Processing (CS458)

Indian Institute of Information Technology Dharwad

Abstract. Almost every NLP pipeline begins with some form of text cleaning, and stopwords removal is arguably the most universally applied step of all. Yet in most projects it gets perhaps five minutes of thought: grab the NLTK list, apply it, move on. This project takes that five-minute decision seriously and asks what actually happens to classification performance when you vary the stopwords strategy systematically. Using the Reuters-21578 newswire corpus as a testbed, we run 15 controlled experiments — five stopwords strategies across three classifiers — and find that the answer is very different depending on which model you are using. The best static configuration (Extended stopwords + SVM) achieves a macro-F1 of 0.9070. A novel corpus-driven adaptive strategy compresses the vocabulary by 87.59% (from 14,794 to just 1,836 features), lifts Naive Bayes F1 by 20.3%, and cuts Logistic Regression training time by 73.9% — all without any predefined word list.

Keywords: stopwords removal, text classification, Reuters-21578, TF-IDF, Naive Bayes, SVM, Logistic Regression, adaptive stopwords.

Contents

1	Survey — Literature Review	3
2	Implementation — Methodology and Codebase	6
3	Analysis — Experimental Results and Interpretation	9
4	Innovation — Adaptive TF-IDF Stopword Strategy	12
	References	17

1 Survey — Literature Review

1.1 Problem Setting and Motivation

If you have spent any time building text classification systems, you have almost certainly removed stopwords without thinking too hard about it. It is the kind of step that gets inherited from tutorial code, applied automatically, and never revisited. This project started from a simple observation: almost nobody in the literature has sat down and measured what stopwords removal actually does to classifier performance in a controlled way, across multiple strategies and multiple models, on the same dataset. The studies that exist are scattered across different tasks, different corpora, and different years, making it difficult to draw clean conclusions.

The dataset chosen for this study is Reuters-21578, a collection of financial newswire articles that has been a standard text classification benchmark since the early 1990s. The core question is: does removing stopwords improve, degrade, or leave unchanged the performance of Naive Bayes, Logistic Regression, and SVM on this corpus, and does the answer change depending on how aggressively you remove?

1.2 Historical Roots of Stopword Removal

The story starts not with machine learning but with library science. Hans Peter Luhn, working at IBM in the late 1950s on automatic document summarisation, was the first person to write formally about the problem of common words [1]. His observation was straightforward: the words that appear most often in any large body of text — *the*, *of*, *and*, *to* — say almost nothing about what a particular document is about. They are the wallpaper of language: always present, contributing little.

Gerard Salton picked up this idea when building the SMART retrieval system at Cornell in the 1970s [2], and it was in the context of information retrieval that the practice became formalised. Salton maintained explicit blacklists of words that should be ignored during indexing, and Van Rijsbergen’s influential textbook [3] spread the convention widely. By the time machine learning-based text classification came along in the 1990s, removing stopwords was already so standard that few people thought to question it.

The theoretical argument for removal is most elegant in a TF-IDF framework. Any word that appears in nearly every document has a document frequency approaching the corpus size, which drives its inverse document frequency (IDF) toward zero. In the resulting TF-IDF vector, that word’s weight is essentially zero regardless of how many times it appears in a given document. In that sense, TF-IDF already does what stopwords removal is trying to do — it just does it implicitly. The practical value of explicit removal is not discriminative power but efficiency: fewer features means smaller matrices, faster training, and lower memory usage, which matters when your vocabulary runs to tens of thousands of tokens.

1.3 The Landscape of Stopword Lists

Not all stopwords lists are designed with the same philosophy, and the differences matter more than they might initially appear.

General-purpose library lists. The NLTK English stopwords list (179 words) and the spaCy equivalent (over 300 words) are by far the most commonly used in practice. Both reflect the conventions established in information retrieval research going back to Van Rijsbergen [3]: articles, auxiliary verbs, prepositions, conjunctions, and common adverbs. They are a reasonable default for most English text, but because they were designed to be broadly applicable, they will always miss words that are common noise in a specific domain.

Minimal lists. A minimal approach keeps only the most obviously uninformative words — typically the articles (*a*, *an*, *the*) and a handful of the most frequent pronouns — and leaves everything else untouched. The rationale is caution: for tasks like sentiment analysis, words that look like function words can carry real meaning. Negation words such as *not* and *never* are technically common, but removing them can completely invert the sentiment of a sentence. For topic-based news classification the concern is less acute, but the minimal strategy is still useful as a conservative baseline.

Extended domain-specific lists. The most practically useful improvement over a general list is to add words that are high-frequency noise within a particular corpus. In Reuters-21578, this means words like *mln* (the standard Reuters abbreviation for “million”), *pct* (percent), *reuter* (a dateline artefact at the start of almost every article), and *said* (appears in nearly every news report regardless of topic). None of these appear on the NLTK list, but none of them tell you anything about whether an article is about earnings, grain prices, or currency markets.

Adaptive data-driven lists. The most intellectually interesting approach is to derive the stopwords list from the corpus itself rather than from any predefined source. Zou et al. explored this for Chinese text using term frequency distributions [15], and Lo et al. used mutual information between tokens and class labels to build corpus-specific lists for retrieval systems [16]. Both papers demonstrate that corpus-derived lists outperform general ones, which is the direct precursor to the adaptive strategy developed in this project.

1.4 Stopword Removal Across NLP Tasks

1.4.1 Text Classification

The impact of stopwords removal on text classification is not uniform — it depends almost entirely on the model being used, a point that the literature establishes clearly even if it rarely measures it in a controlled multi-model setting.

Naive Bayes is the most sensitive model. The reason is structural: MultinomialNB estimates the probability of each word given a class independently, and stopwords — words that appear with roughly equal frequency across all categories — produce near-uniform class-conditional probability estimates. They add noise to the posterior without adding any discriminative signal. McCallum and Nigam [7] and Rennie et al. [8] both documented this sensitivity, and our results

confirm it dramatically: NB F1 improves by 20.3% under the adaptive strategy.

SVMs are a different story entirely. Joachims showed on Reuters-21578 that LinearSVC achieves strong performance without any stopwords removal [6], because the margin-maximisation objective effectively assigns near-zero weights to non-discriminative features. The model already handles noise; explicit removal has little additional effect. Our results show SVM F1 varying by just 0.55 percentage points across all five strategies. Logistic Regression falls between these extremes, showing modest but consistent gains.

Uysal and Gunal’s systematic preprocessing study [9] reached the same conclusion: the optimal preprocessing configuration is task- and classifier-dependent. This finding is precisely the motivation for running all 15 configurations rather than picking a single strategy and assuming it generalises.

1.4.2 Information Retrieval

Stopword removal in information retrieval was historically motivated by storage and speed rather than by retrieval quality [4, 10]. Removing common words from both queries and index reduces disk usage and lookup time without meaningfully affecting recall for most queries. Robertson and Sparck Jones identified the key failure case: when a query is composed almost entirely of common words — the classic example is a phrase search for “*to be or not to be*” — removing stopwords destroys the query entirely [11]. This insight led directly to probabilistic retrieval models like BM25 that weight terms by IDF rather than discarding them outright, and represents a conceptual shift from binary removal to continuous down-weighting.

1.4.3 Topic Modelling

Latent Dirichlet Allocation and similar models are arguably more sensitive to stopwords than any supervised classifier, and the mechanism is direct: the topic-word distributions that LDA learns are shaped by raw token frequencies, so without removal, the most probable words under virtually every topic are function words. Blei et al. acknowledged this in the original LDA paper [12], and Schofield and Mimno’s careful ablation study [13] showed that aggressive removal — including domain-specific terms — consistently improves the coherence and interpretability of discovered topics.

1.4.4 Sentiment Analysis: An Important Counterexample

It would be tempting to conclude from the above that stopwords removal is generally a good idea, but the sentiment analysis literature makes a strong case against that conclusion. Thelwall et al. [22] and Saif et al. [14] both showed that applying standard stopwords lists to sentiment corpora degrades classifier performance. The reason is that function words are not always semantically empty: negation words (*not*, *never*, *hardly*), intensifiers (*very*, *extremely*), and contrastive conjunctions (*but*, *although*) are all present on standard lists yet are among the most important words for determining how positive or negative a text is. Remove them, and you lose critical sentiment-bearing information. For topic-based news classification, this concern is much less pressing — no one would argue that removing *the* and *of* changes the topic of a Reuters

article about grain futures. But it is a strong reminder that stopwords removal is not a universal preprocessing step; it is a design choice that needs to be evaluated for each task.

1.5 Prior Benchmarks on Reuters-21578

Reuters-21578 has been a standard benchmark since its release in 1990 and has attracted considerable attention in the classification literature. Joachims established one of the earliest high watermarks with SVMs achieving micro-F1 of approximately 0.87 [6]. Yang and Liu’s re-examination of text categorisation methods [21] confirmed that SVMs and k -NN were the leading approaches of the classical era, with Naive Bayes trailing noticeably — consistent with our finding that baseline NB F1 is 0.5951 while baseline SVM reaches 0.9058. Sebastiani’s comprehensive survey [17] consolidates results across a wide range of methods and remains a useful reference for understanding where any new result stands historically.

On the stopwords side, Lo et al. [16] measured F1 improvements of 3–8% for Naive Bayes with extended domain-specific lists on 20 Newsgroups, which matches our Reuters result closely (extended stopwords give a 6.7% NB F1 gain). Zou et al. [15] demonstrated that TF-IDF thresholding outperforms static frequency cutoffs on domain-specific corpora, directly motivating the adaptive approach in this project. Sebastiani’s own observation that TF-IDF weighting acts as an implicit stopwords mechanism [17] raises a fair question: if TF-IDF already down-weights common words, why remove them explicitly? The answer, as our experiments confirm, is feature space reduction: the F1 gains for NB and LR come not just from down-weighting but from genuinely shrinking the vocabulary to the tokens that carry signal.

1.6 Identified Research Gaps

Reading through this body of work, three gaps stand out that this project directly addresses.

First, no study has conducted a clean multi-model comparison of static versus adaptive stopwords strategies on Reuters-21578 under identical experimental conditions. Most papers compare one or two static lists on a single model and draw general conclusions.

Second, document length as a moderating variable is almost entirely absent from the literature. Given that Reuters-21578 has very high length variability — the coefficient of variation on document length is 99.7% — and that stopwords removal has a proportionally larger impact on shorter documents, this is an important gap.

Third, few papers explicitly quantify the dimensionality-versus-performance trade-off, meaning how much classification performance is gained or sacrificed per percentage point of vocabulary compression. This is exactly the kind of information a practitioner needs when making a deployment decision, and it is what the analysis section of this project aims to provide.

2 Implementation — Methodology and Codebase

2.1 Overview

The implementation follows a modular pipeline design that keeps the five stopwords strategies cleanly separated from the rest of the system. Data loading, preprocessing, vectorisation, and evaluation are all shared; the only thing that changes between configurations is which word list is applied in the tokenisation step. This makes it straightforward to run all 15 configurations without risking any accidental coupling between them. The full codebase — five Jupyter notebooks, all Python modules, and 106 unit tests — is available at github.com/RohanBabhale1/stopword-impact-analysis.

2.2 Dataset

Reuters-21578 consists of 21,578 newswire articles from 1987, distributed across 22 SGML files and originally annotated with 135 topic categories. After filtering to keep only articles that have at least one topic label and a non-empty body, and restricting to categories with at least 5 training examples so that all classifiers have something meaningful to learn, the working dataset is **10,324 articles across 57 topics**.

Table 1: Reuters-21578 Dataset Summary

Metric	Value
Total / with body / with labels	21,578 / 19,043 / 11,367
Unique topics / topics ≥ 20 docs	120 / 57
Unique tokens in full vocabulary	38,858
Average stopwords ratio (NLTK)	36.74%
Mean document length / median	136 words / 90 words
TF-IDF vocabulary (no removal)	14,794 features

One property worth highlighting is the class imbalance. The *earn* category alone accounts for nearly 39% of labelled documents, while many valid categories have fewer than 30 examples. This is why macro-averaged F1 is reported throughout: it gives each category equal weight regardless of size, which makes it a more honest measure of whether a model actually works across the full label space rather than just being good at the common categories.

2.3 Preprocessing Pipeline

Every document passes through exactly the same five steps regardless of which stopwords strategy is being evaluated. The strategies diverge only at step 4.

1. **Lowercasing.** The full document body is converted to lowercase to collapse surface-form variations like *Earnings* and *earnings*.
2. **Noise removal.** URLs, email addresses, punctuation, and standalone numbers are stripped via regular expressions. This also handles newswire-specific artefacts such as repeated

dashes and ticker-style separators.

3. **Tokenisation.** NLTK’s word tokeniser splits the cleaned text into tokens, correctly handling contractions and punctuation-attached words.
4. **Stopword removal.** Tokens on the active strategy’s list are discarded. For the adaptive strategy, the list is built from training data only (see Section 2.5).
5. **TF-IDF vectorisation.** The surviving token sequence is transformed into a TF-IDF vector using `sublinear_tf=True`. The vocabulary is fitted on the training split only; test documents are transformed under the same vocabulary, with any unseen tokens silently ignored.

The train/test split is 80/20, stratified by topic label with `random_state=42`, ensuring that even rare categories are proportionally represented in both partitions.

2.4 Five Stopword Strategies

Table 2: Five Stopword Strategies Evaluated

Strategy	Size	Description
<code>none</code>	0	No removal; raw bag-of-words baseline
<code>minimal</code>	10	Articles and core function words only: <i>a, an, the, is, of, to, in, it, and, or</i>
<code>nltk</code>	179	Full NLTK English stopwords list
<code>extended</code>	≈200	NLTK plus Reuters-specific tokens: <i>said, mln, pct, reuter, dlrs, cts, shr, avg, revs, ...</i>
<code>adaptive</code>	varies	Corpus-derived: tokens whose mean TF-IDF score across training documents falls below 0.0005

The extended list required a quick manual pass over the top-200 most frequent corpus tokens to identify those that carry no topic-discriminative value. The word *mln* is a good example: it is Reuters shorthand for “million” and appears in articles about earnings, trade figures, loans, and crop yields alike — everywhere that numbers appear, which is almost everywhere. It is never on any general stopwords list, but it belongs on one for this corpus.

2.5 Adaptive TF-IDF Stopword Generation

The adaptive strategy is what makes this project different from a routine comparison of existing lists. The idea is to let the data decide which words are uninformative rather than relying on any external authority.

Given the TF-IDF matrix of the training documents $M \in \mathbb{R}^{n \times |V|}$, the mean TF-IDF score for

each vocabulary token is computed as:

$$\bar{s}(w) = \frac{1}{n} \sum_{i=1}^n M_{i,w} \quad (1)$$

A token whose mean score is very low appears with near-zero TF-IDF weight across most documents — either because it is so common that its IDF is negligible, or because it is so rare that its term frequency contribution is negligible. Either way, it is unlikely to help a classifier distinguish between categories. The adaptive list collects all such tokens:

$$\mathcal{S}_{\text{adaptive}} = \{w \in V \mid \bar{s}(w) < \tau\}, \quad \tau = 0.0005 \quad (2)$$

The threshold was chosen by evaluating NB validation F1 across a range of candidate values on the training fold; 0.0005 is where F1 peaks before starting to decline as genuinely informative tokens begin to get caught. On Reuters-21578, this removes **12,958 tokens**, shrinking the active vocabulary from 14,794 to just **1,836 features** — an 87.59% reduction. Because the list is derived solely from training data, test documents are never seen during its construction. There is no leakage.

2.6 Classifiers and Experimental Protocol

Three classifiers were chosen to span a range of modelling assumptions. Naive Bayes (MultinomialNB, $\alpha = 1.0$) is a generative model that assumes conditional independence between features. It is fast, interpretable, and well-known to be sensitive to noisy features, making it the primary lens for measuring stopwords impact. Logistic Regression (L2 penalty, $C = 1.0$, `lbfgs` solver) is a discriminative linear model that is more robust to feature noise than NB but still benefits from vocabulary reduction at the efficiency level. SVM (`LinearSVC`, $C = 1.0$) maximises the margin between class boundaries, which implicitly down-weights non-discriminative features and makes it the most theoretically robust model to stopwords choice.

All 15 configurations (5×3) were trained and evaluated under identical conditions: same random seed, same split, same TF-IDF parameters. Evaluation covers macro accuracy, macro precision, recall, and F1, plus wall-clock training time on the same hardware.

2.7 Testing and Reproducibility

The 106 unit tests cover all parts of the pipeline: SGML parsing correctness, all five stopwords list generators, the no-leakage property of the adaptive generator, TF-IDF vocabulary fitting and transform behaviour, and metric computation. Every experiment is fully reproducible by setting `random_state=42` and running the provided notebooks in sequence. Trained model pipelines are saved as pickle files in the repository so that inference does not require re-training.

3 Analysis — Experimental Results and Interpretation

3.1 Complete Results

Table 3: All 15 Experimental Configurations (best overall in **bold**)

Strategy	Model	Features	Train (s)	Accuracy	F1
none	NB	14,794	0.086	0.6809	0.5951
none	LR	14,794	11.346	0.8644	0.8430
none	SVM	14,794	1.207	0.9109	0.9058
minimal	NB	14,786	0.072	0.6901	0.6027
minimal	LR	14,786	11.747	0.8659	0.8452
minimal	SVM	14,786	1.170	0.9090	0.9037
nlTK	NB	14,673	0.072	0.7143	0.6298
nlTK	LR	14,673	11.469	0.8712	0.8528
nlTK	SVM	14,673	0.931	0.9099	0.9044
extended	NB	14,669	0.067	0.7177	0.6345
extended	LR	14,669	10.948	0.8726	0.8551
extended	SVM	14,669	0.911	0.9123	0.9070
adaptive	NB	1,836	0.049	0.7603	0.7160
adaptive	LR	1,836	2.960	0.8775	0.8588
adaptive	SVM	1,836	2.120	0.9075	0.9015

3.2 Vocabulary Compression

The first thing that jumps out from the results is how little the static strategies actually compress the vocabulary. Removing 179 NLTK stopwords from a 14,794-token feature space eliminates just 0.82% of features. Adding 25 Reuters-specific tokens (extended) barely changes that figure. The adaptive strategy, by contrast, removes 12,958 tokens for an 87.59% reduction. If feature space size is what you care about, static lists barely move the needle.

Table 4: Vocabulary Compression per Strategy

Strategy	Words Removed	Features	Reduction
none	0	14,794	0.00%
minimal	8	14,786	0.05%
nlTK	121	14,673	0.82%
extended	125	14,669	0.84%
adaptive	12,958	1,836	87.59%

This observation has a practical implication that is easy to miss: applying NLTK stopwords removal and then calling it “dimensionality reduction” is misleading. With a 14,794-token

vocabulary, removing 121 words changes almost nothing about the computational burden. If that is what you are trying to achieve, you need an adaptive approach.

3.3 Classifier-Level Analysis

3.3.1 Naive Bayes

NB is the most dramatic story in this study. Starting at a baseline F1 of 0.5951 with no removal, it climbs steadily through each strategy — 0.6027 with minimal, 0.6298 with NLTK, 0.6345 with extended — before jumping sharply to 0.7160 under the adaptive strategy. That final jump of 20.3% is the largest single improvement observed in any configuration, and it happens because the adaptive strategy is the first one that actually removes enough tokens to make a difference.

The structural reason was described in Section 2.5: when thousands of low-IDF tokens are present, MultinomialNB accumulates near-equal log-probability contributions for every class from each of them, which introduces persistent noise into the posterior calculation. Once those 12,958 tokens are gone, the 1,836 remaining features are overwhelmingly ones that appear with meaningfully different frequencies across topic categories — the signal that NB needs to work well.

3.3.2 Logistic Regression

LR presents a different trade-off. The F1 gains are modest: +0.22% for minimal, +0.98% for NLTK, +1.21% for extended, and +1.58% for adaptive. None of these are headline numbers. What is a headline number is the training time: LR drops from 11.35 seconds with the full vocabulary to 2.96 seconds under adaptive stopwords, a 73.9% speedup. For anyone training LR in a loop — cross-validation, hyperparameter search, repeated experiments — that kind of reduction compounds quickly. The extended strategy gives the best raw F1 (0.8551) but costs 10.95 seconds to train. Adaptive gives the second-best F1 (0.8588, actually slightly higher) in 2.96 seconds. The choice between them is not really a performance question; it is a time question.

3.3.3 SVM

SVM results are nearly flat. The best F1 is 0.9070 (extended) and the worst is 0.9015 (adaptive) — a spread of 0.55 percentage points across configurations that differ by nearly an order of magnitude in vocabulary size. This confirms the theoretical expectation that LinearSVC’s margin-maximisation objective is inherently robust to non-discriminative features. The model learns to ignore them during training; whether they are physically present in the feature vector or not makes little difference to the decision boundary.

The small drop under adaptive (0.43 pp) is worth noting because it is the only case in this study where the most aggressive removal strategy is not at least as good as the baseline. The explanation is that SVM benefits from a handful of tokens that the adaptive filter removes — tokens that have a low mean TF-IDF score across all 57 categories but are occasionally prominent in a rare category and carry just enough signal for SVM’s optimiser to give them nonzero weight. The cost is tiny; the vocabulary reduction is 87.59%.

3.4 Delta F1 Analysis

Table 5: $\Delta F1$ Relative to No-Removal Baseline

Strategy	$\Delta F1$ (NB)	$\Delta F1$ (LR)	$\Delta F1$ (SVM)
minimal	+0.0076	+0.0022	−0.0021
nltk	+0.0347	+0.0098	−0.0014
extended	+0.0394	+0.0121	+0.0012
adaptive	+0.1209	+0.0158	−0.0043

Every NB and LR entry is positive, confirming that some degree of stopwords removal helps both models under all strategies tested. SVM entries hover around zero. The pattern is clean enough to suggest a general rule for this type of corpus: if you are using NB or LR, apply the most aggressive stopwords strategy you can justify; if you are using SVM, the strategy barely matters for accuracy, so choose based on efficiency requirements.

3.5 Performance vs. Feature Compression Trade-off

For NB, the relationship between feature compression and F1 is effectively monotonic: more removal means better performance, consistently, across all five strategies. For LR, the trade-off favours the adaptive strategy once efficiency is included: a 1.58 pp F1 cost in exchange for 73.9% faster training is a good deal in almost any real-world setting. For SVM, F1 is flat across the compression axis, meaning the analyst can choose purely on resource grounds.

3.6 Short vs. Long Document Analysis

Reuters-21578 has unusually high document length variability: the mean is 136 words, the median is 90 words, and the coefficient of variation is 99.7%. A 40-word article and a 500-word article face very different conditions when stopwords are removed. For the short article, removing ten common words eliminates 25% of its content; for the long one, the same ten words are barely noticed.

When the test set is split into short documents (at most 50 words) and long documents (at least 150 words), NB shows the strongest and most consistent gains from stopwords removal in the short-document regime. Removing noise tokens from an already sparse feature vector gives the model a proportionally cleaner signal. Long documents are more robust: even with no removal, there are enough genuine content words to let the classifier work. This suggests that for corpora dominated by short text — tweets, headlines, product reviews — the case for aggressive stopwords removal is even stronger than it appears on Reuters as a whole.

4 Innovation — Adaptive TF-IDF Stopword Strategy

4.1 Motivation

Running the static strategies made one thing very clear: NLTK and extended lists remove less than 1% of the vocabulary. They fix a handful of obvious noise words and leave 99% of the feature space untouched. Yet the adaptive strategy, which removes 87.59% of the vocabulary, either improves or barely changes performance across all three classifiers. This means that roughly 87% of the tokens in a Reuters TF-IDF matrix are not helping — and the standard practice of applying an NLTK list does almost nothing about it.

This raises the question: why not build the stopwords list directly from the data? Earlier work has explored related ideas. Lo et al. used mutual information between tokens and class labels [16]. Zou et al. analysed cross-class term frequency distributions [15]. Forman evaluated chi-square and information gain as feature selection criteria [19]. All of these methods are supervised: they require class label information to score tokens. The approach developed here is different in that it is entirely unsupervised. It needs only the token frequencies themselves, not the labels, which means it can be applied to any corpus before any downstream task, not just classification.

4.2 Algorithm

Let $M \in \mathbb{R}^{n \times |V|}$ be the TF-IDF matrix of the training corpus, where $M_{i,w}$ is the TF-IDF score of token w in document i . The mean TF-IDF score per token is:

$$\bar{s}(w) = \frac{1}{n} \sum_{i=1}^n M_{i,w}, \quad w \in V \quad (3)$$

A token with a very small $\bar{s}(w)$ either appears in almost every document (giving it a near-zero IDF) or appears very rarely (giving it a near-zero overall contribution). In either case, it is unlikely to distinguish between topic categories. The adaptive list is:

$$\mathcal{S}_{\text{adaptive}} = \{ w \in V \mid \bar{s}(w) < \tau \}, \quad \tau = 0.0005 \quad (4)$$

On Reuters-21578 this yields 12,958 tokens. The remaining 1,836 are the words that receive a meaningfully non-zero TF-IDF score somewhere in the training corpus — the genuine content words. The entire procedure has one hyperparameter (τ), uses only training data, and requires no domain knowledge.

4.3 Why Naive Bayes Benefits Most

The 20.3% F1 improvement for NB is the clearest result in this study, and it has a clean theoretical explanation. MultinomialNB computes:

$$\log P(c \mid d) \propto \log P(c) + \sum_{w \in d} \text{tf}(w, d) \cdot \log P(w \mid c)$$

For any token whose class-conditional distribution is near-uniform — that is, $P(w \mid c_1) \approx \dots \approx P(w \mid c_{57})$ — the corresponding log-likelihood terms are approximately equal across all classes. They do not affect the argmax, but they do distort the absolute magnitudes of the

class-conditional probability estimates during training, which shifts the decision boundaries in subtle ways. With 12,958 such tokens removed, the model is estimating probabilities over a much cleaner vocabulary where almost every token genuinely varies across topic categories. The result is sharper, more reliable posterior estimates and a 20.3% F1 gain.

4.4 Why SVM is Largely Unaffected

LinearSVC solves a regularised margin optimisation problem. Tokens that carry little or no discriminative information receive near-zero weights from the optimiser — effectively, the model already ignores them. Explicitly removing them before training changes the input dimensionality but does not substantially alter what the margin boundary looks like once it is found. The 0.43 pp F1 drop under adaptive removal is the one exception: a small number of the 12,958 removed tokens have non-zero but weak discriminative signal for particular rare categories that SVM’s margin objective would otherwise capture. The adaptive threshold catches them because their mean TF-IDF across all 57 categories is small, even though for a few categories it is meaningful. This is a genuine limitation of the approach for SVM, though the magnitude is negligible.

4.5 Comparison with Related Work

Table 6: Comparison of Adaptive Stopword Approaches

Method	Selection Criterion	Key Difference
Lo et al. [16]	Mutual information (token–class)	Supervised; requires class labels
Zou et al. [15]	Cross-class frequency distribution	DF-based; Chinese corpus focus
Forman [19]	Chi-square, information gain	Feature selection framing; supervised
Sebastiani [17]	Implicit via TF-IDF weighting	No explicit list generated
This work	Mean TF-IDF threshold	Unsupervised; language-agnostic; single hyperparameter

The key distinction is the absence of class label supervision. Because this method scores tokens on their marginal TF-IDF distribution alone, it can be applied to any corpus at any point in the pipeline, including before labels are available. That makes it more generally applicable than the supervised alternatives.

4.6 Quantitative Summary

Table 7: Adaptive Strategy: Gains over No-Removal Baseline

Model	F1 (none)	F1 (adaptive)	Δ F1	Speedup
Naive Bayes	0.5951	0.7160	+20.3%	1.8×
Logistic Regression	0.8430	0.8588	+1.9%	3.8×
SVM	0.9058	0.9015	−0.5%	0.6×

The adaptive strategy gives the best NB F1 of any configuration tested (0.7160), the best LR F1 (0.8588, marginally ahead of extended), and an LR training speedup of 3.8×. For SVM the story is reversed: a 0.5% F1 dip and a slight training overhead from the two-pass vectoriser design. In any deployment where NB or LR is the model of choice, adaptive is the clear winner on both accuracy and speed.

4.7 Limitations and Future Directions

The adaptive strategy works well on Reuters-21578, but it comes with limitations that are worth being clear about.

Threshold sensitivity. The value $\tau = 0.0005$ was chosen by grid search on the Reuters training fold. A different corpus — one with longer documents, a different vocabulary size, or a different domain — will likely need a different threshold. One direction to make this more automatic would be to detect the natural elbow in the sorted mean TF-IDF distribution, where the curve transitions from a dense cluster of near-zero scores to a sparser region of genuinely informative tokens. This would eliminate the need for manual tuning entirely.

Rare category sensitivity. The threshold is computed over the mean TF-IDF score across all training documents, pooled across all 57 categories. For a token that is highly specific to a category with only 20 or 30 training examples, its mean score across the full training set will be small even if it is very discriminative within that category. The adaptive filter could therefore inadvertently remove tokens that matter for rare classes. A per-class mean TF-IDF check — flag a token as a stopword only if its score is low across every class individually — would mitigate this risk. This is an important extension for any corpus with severe class imbalance.

Transformer and embedding-based pipelines. BERT and similar models [18] operate on subword tokens produced by byte-pair encoding, not full words, and they do not use TF-IDF weighting at all. The mean TF-IDF criterion does not transfer directly to these pipelines. An analogous method based on mean attention weights or token importance scores from the transformer’s self-attention mechanism could serve the same purpose, but would require a different implementation. Exploring this is a natural extension of the present work.

Multi-label evaluation. Reuters-21578 is inherently a multi-label dataset: a single article can belong to multiple topic categories simultaneously. This project evaluates the single-label

case for simplicity and comparability with historical benchmarks. Extending to a full multi-label setup using OneVsRest or classifier chains, and measuring how stopwords strategies affect per-label precision and recall, would give a more complete picture of the adaptive strategy’s behaviour. That evaluation, alongside replication on 20 Newsgroups and AG News, is planned as follow-on work.

Two-pass vectoriser overhead. The adaptive list requires fitting the TF-IDF vectoriser once to compute the mean scores, generating the list, and then re-fitting on the filtered vocabulary. For Reuters-scale data this is negligible, but for very large corpora this two-pass design adds meaningful overhead. An online estimation of the mean TF-IDF score during a single pass would make the approach more scalable.

References

- [1] Luhn, H.P. (1958). The automatic creation of literature abstracts. *IBM Journal of Research and Development*, 2(2), 159–165.
- [2] Salton, G. (1971). *The SMART Retrieval System: Experiments in Automatic Document Processing*. Prentice-Hall.
- [3] Van Rijsbergen, C.J. (1979). *Information Retrieval* (2nd ed.). Butterworths.
- [4] Fox, C. (1990). A stop list for general text. *ACM SIGIR Forum*, 24(1–2), 19–21.
- [5] Lewis, D.D., Yang, Y., Rose, T., & Li, F. (2004). RCV1: A new benchmark collection for text categorization research. *Journal of Machine Learning Research*, 5, 361–397.
- [6] Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. *Proceedings of ECML 1998*, 137–142.
- [7] McCallum, A., & Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. *AAAI Workshop on Learning for Text Categorization*, 41–48.
- [8] Rennie, J.D., Shih, L., Teevan, J., & Karger, D.R. (2003). Tackling the poor assumptions of Naive Bayes text classifiers. *Proceedings of ICML 2003*, 616–623.
- [9] Uysal, A.K., & Gunal, S. (2014). The impact of preprocessing on text classification. *Information Processing & Management*, 50(1), 104–112.
- [10] Wilbur, W.J., & Sirotkin, K. (1992). The automatic identification of stop words. *Journal of Information Science*, 18(1), 45–55.
- [11] Robertson, S.E., & Sparck Jones, K. (1976). Relevance weighting of search terms. *Journal of the American Society for Information Science*, 27(3), 129–146.
- [12] Blei, D.M., Ng, A.Y., & Jordan, M.I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3, 993–1022.
- [13] Schofield, A., & Mimno, D. (2016). Comparing apples to apple: The effects of stemmers on topic models. *Transactions of the Association for Computational Linguistics*, 4, 287–300.
- [14] Saif, H., He, Y., Fernandez, M., & Alani, H. (2014). On stopwords, filtering and data sparsity for sentiment analysis of Twitter. *Proceedings of LREC 2014*, 810–817.
- [15] Zou, F., Wang, F.L., Deng, X., Han, S., & Wang, L.S. (2006). Automatic construction of Chinese stop word list. *Proceedings of the 5th WSEAS International Conference on Applied Computer Science*, 1009–1014.
- [16] Lo, R.T., He, B., & Ounis, I. (2005). Automatically building a stopword list for an information retrieval system. *Proceedings of the 5th Dutch-Belgian Information Retrieval Workshop (DIR)*, 17–24.

- [17] Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1–47.
- [18] Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [19] Forman, G. (2003). An extensive empirical study of feature selection metrics for text classification. *Journal of Machine Learning Research*, 3, 1289–1305.
- [20] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *Proceedings of ICLR 2013*.
- [21] Yang, Y., & Liu, X. (1999). A re-examination of text categorization methods. *Proceedings of ACM SIGIR 1999*, 42–49.
- [22] Thelwall, M., Buckley, K., & Paltoglou, G. (2012). Sentiment strength detection for the social web. *JASIST*, 63(1), 163–173.